

BUT Informatique - R3.02

Conception d'algorithmes corrects et efficaces

Méthodes : algorithmes gloutons

J. Christian Attiogbé

Novembre 2022



Contenu prévisionnel

Algo de Prim - arbre recouvrant (spanning tree)

On considère des **graphes ou réseaux de nœuds** qui sont connectés de façon **bidirectionnelle** ou bien de façon **unidirectionnelle (orientée)**.

Un problème dont la solution est apportée par l'algorithme de PRIM est basée sur un graphe avec des liens bidirectionnels.

Chaque **lien a un coût** qui est un entier positif. Tout nœud (ou site) peut atteindre tout autre nœud en empruntant une **suite de liens**.

Algorithme recherché (PRIM)

On recherche un acheminement d'une information entre deux nœuds, tel que **la somme des coûts des liens empruntés pour atteindre l'ensemble des sites soit minimale**.



Des applications

Variante de l'algorithme avec graphes unidirectionnels : DIJKSTRA.

Applications

Il existe de nombreux problèmes pratiques résolus sur la base de ces algorithmes fondamentaux :

- câblage d'un ensemble de bâtiments,
- système optimal d'irrigation de parcelles,
- acheminements dans les réseaux,
- acheminements dans les transports,
- etc

Graphes : unidirectionnels, bidirectionnels

On considère des graphes ou réseaux de nœuds qui sont connectés de façon bidirectionnelle ou bien de façon unidirectionnelle (orientée).

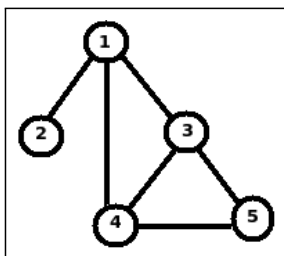


Figure: Un graphe non orienté, connexe, non pondéré

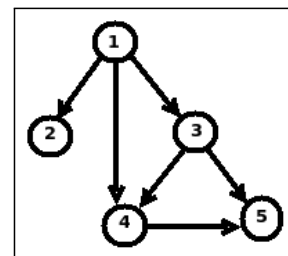


Figure: Un graphe unidirectionnel, non pondéré

Graphes : arcs pondérés

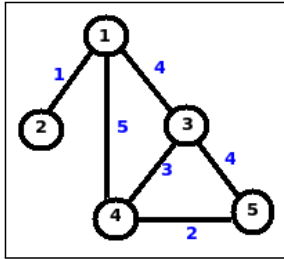


Figure: Un graphe non orienté, connexe

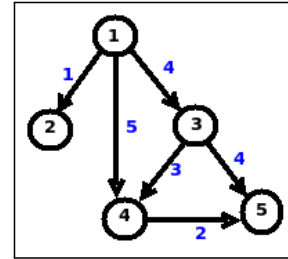


Figure: Un graphe unidirectionnel

Étude de l'algorithme de P_{PRIM}

Soit $G = (N, A, P)$ un graphe non orienté

N : ensemble des sommets ou nœuds est appelé

$A \subseteq N \times N$: ensemble des *arêtes* appelé

G est valué/pondéré par une fonction P (poids)

$$P : A \rightarrow \mathbb{N}$$

G connexe : il existe une chaîne reliant toute paire de sommets distincts de G .

$T = (N, A, P)$ un arbre (*Tree*) non orienté

valué/pondéré sur les entiers positifs.

pondus d'un arbre = somme des valeurs de ses arêtes (ou branches).

un arbre de k sommets possède $(k-1)$ arêtes,

une chaîne élémentaire unique relie toute paire de sommets distincts d'un arbre.

Arbre de recouvrement d'un graphe

Arbre de recouvrement (spanning tree) de $G = (N, A, P)$, un arbre (non enraciné), incluant tous les sommets de G .

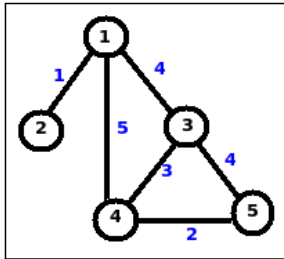


Figure: Un graphe non orienté, connexe

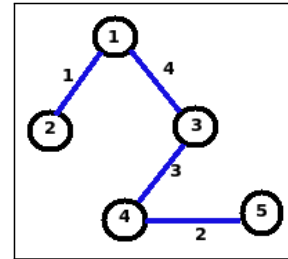


Figure: Un arbre de recouvrement associé au graphe de la fig.

Arbre de recouvrement de G de **plus faible poids** est appelé arbre de recouvrement **de poids minimal (arpm)** ou arbre sous-tendant de poids minimal (**astm**).

Arbre de recouvrement d'un graphe

On sait montrer (**Propriété 1**) qu'un graphe G connexe, admet au moins un **arpm**, et qu'un arbre est son propre **arpm**.

En référence à un graphe $G = (N, A, P)$, on dit d'un arbre T qu'il est **prometteur** s'il est un **sous-arbre d'un arpm**.

On sait démontrer (**Propriété 2**) que l'adjonction à un arbre prometteur $T = (N', A', P')$ de $G = (N, A, P)$, de l'une des **arêtes mixtes** de G de valeur minimale, **résulte à un nouvel arbre prometteur**.

✂ **Travail partiellement réalisé ; progression à partir d'un noeud !**

Principe de l'algorithme de PRIM

- 1 on fixe **un sommet source** $\Rightarrow$ ce sommet est un arbre prometteur (sommet 1 dans exemple) ;
- 2 on fait **grossir l'arbre prometteur** (partant de la source) en lui adjoignant **une nouvelle arête** (et donc un nouveau sommet aussi) du graphe G que l'on veut recouvrir.
on n'ajoute que une **arête mixte** (= constituée d'un sommet pré-existant (sommet interne) dans l'arbre, et d'un sommet n'en faisant pas partie (sommet externe)).
- 3 jusqu'à avoir tous les noeuds de G.

👉 comment déterminer quelle arête de G peut/doit être ajoutée dans un arbre prometteur pour que le nouvel arbre résultant soit lui aussi prometteur ?

On adopte le *principe de la course en tête*, pour construire l'algorithme.

Principe de la 'Course en tête' (Glouton)

Principe de la 'Course en tête'

Le principe est de considérer le **critère d'optimalité dès le départ** d'un algorithme qui procède par exemple à des **choix successifs, étape par étape**, pour la suite d'un traitement.

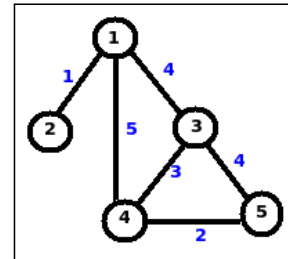
Ainsi **à chaque étape, le choix le plus optimal est effectué** ; et ainsi de suite lors des étapes suivantes.

Cela peut être lors du parcours d'un arbre/graphe pour le choix du noeud suivant, la tâche suivante lors de l'ordonnancement de tâches, etc

Construction de l'algorithme : choix structure des données

Graphe non orienté : $P : 1..n \times 1..n \rightarrow \mathbb{N}_1$

	1	2	3	4	5
1	M	1	4	5	M
2	1	M	M	M	M
3	4	M	M	3	4
4	5	M	3	M	2
5	M	M	4	2	M



Noeuds du graphe : $N = 1..n$ (externes), puis vont rejoindre l'arbre

Noeuds de l'arbre : $N' = 1..n$ (en commençant par 1, puis ...)

Les arêtes mixtes (de externe vers interne) dans $(N \times N')$, mais la source (1) n'est pas relié à un autre noeud.

$$aM : 2..n \rightarrow 1..n$$



Construction de l'algorithme

Les arêtes de G :

$\{(2, 1), (1, 2), (1, 4), (4, 1), (1, 3), (3, 1), (3, 4), (4, 3), (4, 5), (5, 4), (3, 5), (5, 3)\}$

Les arêtes internes dans l'arbre :

$$aI : 2..n \rightarrow 1..n$$

Initialement $N' = \{1\}$, donc $N = 2..n$

Les sommets externes et les sommets internes forment une partition de N ($1..n$)

Initialement, $N' = \{1\}$, $aI = \{\}$,

et donc tous les autres sommets pointent vers (1) :

$$aM = (2..n) \times \{1\}$$

$$= \{(2, 1), (3, 1), (4, 1), (5, 1)\}$$

Construction de l'algorithme

Postcondition:

$T = (N', A', P')$ est un arbre prometteur de $G = (N, A, P)$ et $\text{card}(A') = \text{card}(N') - 1 = \text{card}(N) - 1 = n - 1$.

Recherche d'un invariant

Invariant:

$T = (N', A', P')$ est un arbre prometteur de $G = (N, A, P)$ et $N' \subseteq N$

Recherche d'une condition d'arrêt

Condition arrêt :

$$\text{card}(A') = n - 1$$

Construction de l'algorithme

Recherche de la progression

Progression :

Ajout à T d'une arrête mixte (s_e, s_i) , avec s_e dans N et s_i dans N'

$$aI = aI \cup \{(s_e, s_i)\}$$

avec $P(s_e, s_i) = \min(\{(nn, s_i) \text{ appartenant à } aM\})$ **course en tête**

Expression de Terminaison

Terminaison :

$$(n - 1) - \text{card}(N')$$

décroît à chaque progression.

Construction de l'algorithme de PRIM

Constants

$n \in \mathbb{N}_1$ et $n = \dots$ et $P \in 1..n \times 1..n \rightarrow \mathbb{N}_1$ et
 $P = P^T$ {transposé} et $P = [\dots]$ et $EstConnexe(G)$
 { donc $P(e,i) = P(i,e)$ } { $P = ((e,i) \mapsto p_k), \dots$ }

Variables

$areteI \in 2..n \rightarrow 1..n$ et
 $areteM \in 2..n \rightarrow 1..n$ et
 $dom(areteI) \cup dom(areteM) = 2..n$ et
 $dom(areteI) \cap dom(areteM) = \emptyset$ {partition}

debut

Precondition: ...

$areteI \leftarrow \emptyset$;

$areteM \leftarrow 2..n \times \{1\}$; {Toutes les aretes ont 1 comme arête père}

Construction de l'algorithme

Pour $e \in 2..n$ **faire** {parcours des noeuds du graphe}

soit (e, i) **tel que**

$(e, i) \in areteM$ et $P(e, i) = \min(\{P(l, i) \mid (l, i) \in areteM\})$

{ (e, i) est l'arête mixte de valeur minimale }

alors

$areteI \leftarrow areteI \cup \{(e, i)\}$; {ajout dans l'arbre}

$areteM \leftarrow areteM - \{(e, i)\}$; {retrait de aM}

Pour $e' \in dom(aM)$ {parcours des noeuds du graphe}

{recherche e' sommet externe le plus proche du nouv interne e }

Si $(P(e', e) < P(e', areteM(e')))$

alors $areteM(e') \leftarrow e$ {ajout de e vers e' }

finSi

finPour

finSoit

finPour

Postc : la postcondition

Fin

Résultat du déroulement

$$aI = \{(2,1)\}$$

$$aI = \{(2,1), (3,1)\}$$

$$aI = \{(2,1), (3,1), (4,3)\}$$

$$aI = \{(2,1), (3,1), (4,3), (5,2)\}$$

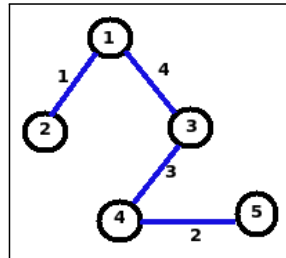


Figure: L'arbre de recouvrement associé au graphe

Conclusion

Algorithme de Prim : un classique sur les graphes

Très utilisé en pratique

(Exemple en réseau : connexion entre routeurs ; etc)

S'entraîner avec ces algorithmes classiques

Conclusion générale

Bilan du cours

- Initiation à la **complexité des algorithmes** ; chasser les **coûts**.
- Plusieurs **méthodes rigoureuses de construction correcte** des algorithmes
- **Spécification** des algorithmes/programmes - Invariant
- Triplet de HoARE ; contrat
- Choix des structures de données → **Conception**
- Construction rigoureuse des **boucles/programmes**

- AC21.03 : Adopter de bonnes pratiques de conception et de programmation
- AC22.01 : Choisir des structures de données complexes adaptées au problème
- AC22.02 : Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)
- AC22.04 : Évaluer l'impact environnemental et sociétal des solutions proposées

AC21.03	✓	AC22.01	✓	AC22.02	✓	AC22.04	✓
---------	---	---------	---	---------	---	---------	---



Merci

MERCI pour votre écoute !